Spring Boot Annotations Guide

9 must-know annotations every Java developer should master

1. `@RequestMapping`
2. `@GetMapping`
3. `@SpringBootApplication`
4. `@RestController`
5. `@Component & @Bean`
6. `@Repository & @Entity`
7. `@PathVariable & @RequestParam`
8. `@Service & @Autowired`
9. `@RequestBody`

@RequestMapping

Maps HTTP requests to handler methods
at class or method level.

```java
@RestController
@RequestMapping("/api/users")
public class UserController {
    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.getUserById(id);
    }
}
```

- Used to define the base URL for a controller or map specific endpoints.
- Supports GET, POST, PUT, DELETE, PATCH and more.

GET

@GetMapping

Handles HTTP GET requests and returns the requested data.

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    @GetMapping
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }
}
```

- Shortcut for @RequestMapping(method = RequestMethod.GET)
- Used to fetch data from the server.

@SpringBootApplication

Marks the main class to start the Spring Boot application.

```java
@SpringBootApplication
public class DemoApplication {
  public static void main(String[] args) {
    SpringApplication.run(DemoApplication.class, args);
  }
}
```

- @Configuration Allows Spring to register beans.
- @EnableAutoConfiguration Enables auto configuration.
- @ComponentScan Scans components in the package.

@RestController

Marks a class as a REST controller where every method returns data instead of a view.

```java
@RestController
@RequestMapping("/api")
public class UserController {
  @GetMapping("/users")
  public List<String> getAllUsers() {
    return userService.getAllUsers();
  }
}
```

- @RestController Combination of @Controller and @ResponseBody.
- Perfect for building RESTful Web Services.

@Component & @Bean

Used to register Spring-managed components and create beans manually.

```
@Component
public class EmailService {
  public void sendEmail(String msg) { }
}

@Configuration
public class AppConfig {
  @Bean
  public RestTemplate restTemplate() {
    return new RestTemplate();
  }
}
```

- @Component Marks the class as a Spring-managed component. Auto-detected via scanning.
- @Bean Registers a method's return object as a bean in the Spring context.

@Repository & @Entity

Used for database operations and defining database entities.

```java
@Entity
public class User {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;
  private String name;
}

@Repository
public interface UserRepository extends JpaRepository<User, Long> { }
```

- @Entity Marks a class as a database entity. Each object maps to a table.
- @Repository Marks the interface for the database access layer with CRUD operations.

@PathVariable & @RequestParam

Used to capture data from the URL
in different ways.

```java
// Using @PathVariable
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) { }

// Using @RequestParam
@GetMapping("/users")
public List<User> getUsers(
    @RequestParam int page, @RequestParam int size) { }
```

- @PathVariable Extracts values from the URL path. e.g. /users/101 id = 101
- @RequestParam Extracts values from query parameters. e.g. ?page=1&size=10

@Service & @Autowired

Used for business logic and dependency injection.

```java
@Service
public class UserService {
  public List<User> getAllUsers() {
    return userRepository.findAll();
  }
}

@RestController
public class UserController {
  @Autowired
  private UserService userService;
}
```

- **@Service**
  Marks the class as a service-layer component holding business logic.
- **@Autowired**
  Injects the dependent object automatically. Core to Dependency Injection.

@RequestBody

Maps the HTTP request body to a Java object.
Mostly used in POST / PUT requests.

```java
@PostMapping("/users")
public User createUser(@RequestBody User user) {
  return userService.save(user);
}

// Example JSON Request
{ "name": "Kamlesh Koli",
  "email": "kamlesh@example.com" }
```

- Purpose: Binds the incoming JSON data to a Java object.
- Key Point: Spring auto-converts JSON into a Java object using HttpMessageConverter.